

Archive de session — 10 octobre 2026

Session du 10/10 (partie « soir »), de la demande de passage du script de clonage en **v2.4** jusqu'au « bye bye ». Un seul chantier : le script `zeta_backup_toshiba.sh` reçoit un **garde-fou sur le sens du clonage** et une **restauration dans le sens inverse** (clone externe → disque de travail), avec tests sans aucune écriture sur un disque, documentation, commits, propagation sur les quatre branches, Handoff et wiki.

Capture brute de la session : absente (aucun processus script parmi les ancêtres du terminal de Claude Code). Des journaux de capture du jour existent mais datent de sessions antérieures. La transcription automatique de Claude Code reste disponible ; sa conversion par `zeta-convert-jsontomd` est proposée en fin de session.

Ce document ne contient ni extrait de code, ni adresse réseau, ni identifiant de volume.

1. Reprise et vérifications préalables

Consigne de hprzeta : avant toute modification du script, vérifier que les trois derniers commits (`bcf7f9c`, `3681f19`, `01bd5b7`) sont propagés sur les quatre branches et qu'aucun worktree de fusion n'est resté ouvert ; sinon, le signaler d'abord.

Contrôle	Résultat
Commits présents sur <code>Riemann_Lab_IA</code> , <code>Riemann_Lab_C</code> , <code>Riemann_Lab_Test</code> , <code>main</code> (distant)	oui, les trois
Worktrees ouverts	aucun (un seul, le dépôt principal)
Dépôt local	propre sur <code>Riemann_Lab_IA</code>
Processus du script en cours	aucun
Racine en cours	disque de travail (Seagate)

2. Cahier des charges de la v2.4

Consigne de hprzeta : (A) un garde-fou sur le sens du clonage ; (B) une option de **restauration** dans le sens inverse, utile si le disque de travail tombe en panne ; (C) des tests sans rien écrire ; (D) la documentation. Règles impératives : identifier les disques par **LABEL** (jamais par `sdx`), sauvegarde `.bak` avant toute modification, `cp` et non `tee`, **accord requis** avant toute écriture sur un disque et avant tout commit, exclusion du travail SAP conservée dans les deux sens, tableau d'avancement après chaque étape.

3. Déroulé par étapes

#	Étape	Résultat
0	Vérifications préalables	OK, rien à signaler
1	Lecture complète du script (438 lignes)	fonctions repérées, adaptations listées
2	Sauvegarde, puis garde-fou de sens (<code>require_root_label</code>)	refus testé en simulation (label simulé)
3	Option 8 (restauration) : garde-fous B1/B2, dry-run automatique, rsync inverse, option <code>--dry-run</code>	refus testés ; verrou temporaire sur l'écriture réelle
4	Post-restauration : <code>fstab</code> et GRUB inversés, <code>grub-install + efibootmgr</code> si l'EFI est vide, contrôles finaux, démontage garanti	testé par simulation
5	Option 9 (disque neuf)	refus testés (sans disque réel)
5b	Option 10 et <code>--only</code> : restauration d'un fichier ou dossier précis	refus testés
6	Tests : <code>bash -n</code> , refus par simulation ; <code>shellcheck</code> absent du poste	OK sauf <code>shellcheck</code>
7	Documentation : en-tête et menu v2.4, rapport de clonage, PDF, mémoire	fait
8	<code>git status</code> et <code>git diff --stat</code> , audit des lignes ajoutées, arrêt avant commit	aucune correspondance (secrets, adresses)

Points de conception retenus :

- Le garde-fou de sens exige la racine du disque de travail pour toute écriture sur le clone ; il est commun au clonage, à la réparation du boot et aux deux fonctions de réparation.
- Les fonctions de réparation (`fstab`, GRUB, vérification de bootabilité) sont devenues **communes aux deux sens** : une table de variables source/cible remplace les labels écrits en dur.
- Le démontage garanti (`trap EXIT`) ne démontait que ses propres montages ; il a fallu lui apprendre à **ne démonter qu'à partir d'un indice**, sinon la réparation de GRUB aurait démonté les montages de la restauration en cours.
- La restauration refuse si la cible est la racine en cours (contrôle par label **et** par périphérique), si source et cible sont sur le même disque physique, ou si les partitions cibles ne sont pas toutes sur le même disque.

4. Question de hprzeta : « je n'ai pas de disque neuf »

Question : comment vérifier que le sens inverse marche sans disque neuf ? peut-on restaurer un fichier de test ? Puis : l'étape « disque neuf » exige-t-elle un disque maintenant ?

Réponse retenue (trois niveaux) : (1) dry-run complet au démarrage sur le clone (cible montée en lecture seule) ; (2) **restauration réelle d'un fichier de test** : ajout d'un mode ciblé (option 10, `--only`) qui n'écrit que le chemin demandé, sans suppression ; (3) test de l'option 9 sur un

fichier image (optionnel), grâce à une variable de test qui change les labels et réduit les tailles. L'option 9 est seulement **écrite** maintenant ; elle ne s'exécute que le jour où elle est lancée avec un disque neuf branché. Ce qui ne sera jamais testé avant un vrai besoin : grub-install sur un disque réellement vide.

Décision de hprzeta : « go pour enchaîner les étapes 5, 5b, 6, 7 et 8 ».

5. Commit et push

Commit	Contenu	Branches
b58742f	script v2.4 + section « Script v2.4 » du rapport de clonage + PDF	Riemann_Lab_IA poussée ; Riemann_Lab_C en fast-forward

Audit avant push : aucun secret, aucune adresse. Aucun --force.

6. Clone réel avec le script v2.4

Consigne de hprzeta : option A, rafraîchir le clone pour y copier le script v2.4 et un dossier de test. Le clone demande sudo et un « oui » tapé : **c'est hprzeta qui l'a lancé.**

Dry-run relu avant le « oui » : 1 111 changements dont **21 suppressions côté clone**, toutes jetables (une sauvegarde de fstab, 10 fichiers de session de Claude Code, 8 journaux npm, 2 fichiers de cache Firefox). Aucun fichier créé côté clone pendant un démarrage dessus, aucune ligne SAP, aucun changement sur la partition de données.

Résultat : rsync des trois partitions, correction du fstab et de GRUB, puis **8 contrôles de bootabilité sur 8 OK** (grub.cfg : 27 occurrences de l'identifiant du clone, 0 de l'identifiant du disque de travail). C'est le premier passage **réel** du code refactoré en mode clonage. Vérifié ensuite en lecture seule : le script v2.4 et le dossier de test sont bien sur le clone. L'avertissement file has vanished (code 24) est bénin : fichiers temporaires disparus pendant la copie.

7. Constat sur les dossiers SAP

En vérifiant le clone, j'ai constaté que Documents/SAP et Documents/SAP_import **existent sur le clone**, alors que mon texte affirmait « SAP non récupérable depuis le Toshiba ».

Dossier	Clone	Disque de travail
Documents/SAP	41 fichiers, 19 Mo	4 089 fichiers, 4,2 Go
Documents/SAP_import	4 048 fichiers, 4,2 Go (le plus récent : 30/07)	absent

Explication : l'exclusion SAP* (v2.1) empêche toute nouvelle copie, mais, sans --delete-excluded, ne supprime pas l'existant : le clone garde une **ancienne copie antérieure à l'exclusion**. Seuls des comptages ont été relevés, aucun contenu lu. Rien n'a été supprimé (irréversible, et peut-être l'unique copie de l'ancien nom).

Options proposées : A (laisser et corriger la documentation), B (purger après comparaison), C (autoriser la restauration du SAP). **Décision de hprzeta : A.**

Corrections : section « Limites » du rapport (+ PDF), trois messages du script (en-tête, avertissement de l'option 8, refus de l'option 10), Handoff local, mémoire. Les messages du script n'ont été modifiés qu'**après la fermeture du menu** (règle : ne jamais modifier le script pendant qu'il tourne). Commit bb2b5d8, poussé sur Riemann_Lab_IA et Riemann_Lab_C (fast-forward). La copie du script présente sur le clone est celle d'avant cette correction (messages seulement) : le prochain clonage la mettra à jour.

8. Propagation sur main et Riemann_Lab_Test

Les deux branches ont des commits propres (61 et 126), donc pas de fast-forward : **fusion de IA dans des worktrees isolés**, après vérification de l'état distant.

Branche	Avant	Après	Contrôle
main	ccd9400	1c3bf29	arbre identique à Riemann_Lab_IA, 0 conflit
Riemann_Lab_Test	64bc0db	7b59bec	arbre = IA + 2 fichiers d'expérience, 0 conflit

Audit avant chaque push : 0 résultat ; poussée simple (fast-forward côté distant), sans --force. Worktrees et branches temporaires supprimés ; docs/ inchangé (le site GitHub Pages n'est pas reconstruit).

9. Handoff local et wiki

Consigne de hprzeta : « go pour mettre à jour le Handoff », puis « go wiki ».

- **Handoff local** (hors dépôt) : bloc de reprise mis à jour (v2.4, clone fait, constat SAP, propagation faite, prochaine action).
- **Wiki** (public) : nouvelle entrée du JOURNAL.md et nouveau résumé « REPRENDRE ICI » dans le Handoff.md, **sans détail d'infrastructure**, sans nom de client ; audit : aucune correspondance. Commit wiki c423c89, poussé sur master. Un fichier non suivi préexistant n'a pas été ajouté.
- **Mémoire** corrigée (« non commitée » devenu les commits réels).

10. Ce qui reste à faire

#	Action	Responsable
1	Supprimer le dossier de test du disque de travail (il est déjà sur le clone)	hprzeta
2	Démarrer sur le clone (bande rouge) ; --dry-run des options 8 et 10 ; coller les sorties	hprzeta, puis analyse
3	Restauration réelle du dossier de test par l'option 10 (après lecture du dry-run) ; vérifier le fichier	hprzeta

#	Action	Responsable
4	Mettre à jour le Handoff local avec le résultat du test	Claude
5	Facultatif : installer <code>shellcheck</code> ; remettre à niveau les branches locales <code>Riemann_Lab_C</code> , <code>main</code> , <code>Riemann_Lab_Test</code>	hprzeta / Claude

Jamais testé en réel : `rsync` inverse, `dry-run` des options 8 et 10, `grub-install` et `efibootmgr` (ils ne seront éprouvés qu'au premier remplacement de disque). **Aucune restauration complète (option 8) sans accord explicite de hprzeta**. Restent aussi ouverts les points du 06/10 : rotation du jeton DNS dynamique, masquage éventuel d'identifiants de disque déjà publics, régénération des PDF des anciennes archives.

11. Clôture : « bye bye »

Capture brute du terminal : **absente** pour cette session (aucun processus script actif) ; il n'y a donc rien à arrêter. Pour la prochaine fois : lancer `~/projet_zeta/scripts/capture.sh` **avant** `claude`. La transcription automatique de Claude Code (un fichier `.jsonl` par session) reste disponible ; `zeta-convert-jsontomd` peut la convertir en Markdown masqué (mode 600, ignoré par git, jamais commité sans demande).

Archive de session du 10/10/2026 — rédigée à la demande de hprzeta (« bye bye »).